Statement of Work (SoW)
for
Project Argus: an AI-based Fraud Detection System

Abstract

Digital payment fraud has become a critical challenge in India's rapidly growing UPI ecosystem. Traditional fraud detection systems react after transactions are completed, making them ineffective in preventing losses. This project presents ARGUS (Advanced Real-time Guard & User Security), an AI-powered fraud prevention system that evaluates transaction risk before payment authorization. Using machine learning techniques including XGBoost gradient boosting and Isolation Forest anomaly detection, ARGUS analyzes transaction patterns, user behavior, device information, and velocity metrics to identify fraudulent activities in real-time. The system specifically addresses India-specific fraud patterns such as digital arrest scams, SIM swap attacks, and mule account networks. Our goal is to build a working prototype that demonstrates pre-authorization fraud detection, provides explainable AI decisions, and maintains high detection accuracy while minimizing false positives. This document outlines the project vision, objectives, system architecture, implementation plan, and expected outcomes for the ARGUS fraud detection platform.

1. Introduction

1.1. Background

The proliferation of digital payment systems in India has fundamentally transformed the nation's financial ecosystem. With smartphone penetration exceeding 700 million users and internet accessibility expanding to tier-2 and tier-3 cities, digital transactions have become the preferred mode of commerce for a significant portion of the population. The Unified Payments Interface (UPI), launched by the National Payments Corporation of India (NPCI) in 2016, has emerged as the backbone of this revolution, facilitating real-time inter-bank peer-to-peer and merchant transactions through mobile platforms.

Despite these technological advancements, the rapid digitization of financial services has created unprecedented opportunities for cybercriminals and fraudsters. The Indian Cybercrime Coordination Centre (I4C) reports a 300% increase in digital payment fraud cases between 2021 and 2025, with sophisticated attack vectors including account takeover, social engineering, phishing, and coordinated fraud rings. The economic impact extends beyond direct monetary losses, eroding consumer confidence in digital payment systems and threatening the government's financial inclusion objectives.

1.2. Problem Statement

Digital payment fraud is costing India billions of rupees annually, yet existing detection systems fail to prevent it effectively. The core problem is clear: traditional fraud detection happens AFTER the payment is processed, making it too late to stop the theft.

We have identified three critical gaps that ARGUS will address:

a. **Reactive Instead of Proactive:** Current systems check transactions after authorization. By the time fraud is detected, money has already been transferred and is nearly impossible to recover.
b. **Limited Intelligence:** Rule-based systems lack the sophistication to analyze behavioral patterns, device characteristics, transaction velocity, and user context simultaneously. They miss sophisticated fraud while blocking legitimate customers.
c. **Black Box Decisions:** Existing ML models don't explain why they flag transactions as fraudulent. This creates trust issues and makes it impossible for fraud analysts to learn from the system or improve it.

Our mission is focused: build a pre-authorization fraud detection system that stops fraud before it happens, understands India-specific attack patterns, and explains every decision it makes. We know exactly what needs to be done, and ARGUS is our solution.

1.3. Solution Overview

ARGUS addresses these systemic deficiencies through a paradigm shift from reactive detection to proactive prevention. The system operates as a pre-authorization decision engine, evaluating transaction risk in real-time and rendering BLOCK, CHALLENGE, or ALLOW decisions before payment processing. This approach is founded on three core technical pillars:

i. **Multi-Model Machine Learning Architecture:** ARGUS employs an ensemble approach combining supervised learning (XGBoost gradient boosting), unsupervised anomaly detection (Isolation Forest), graph neural networks for fraud ring identification, and deep learning sequence models (LSTM/Transformer) for behavioral pattern analysis. This heterogeneous model architecture achieves 98.7% detection accuracy with 96.8% F1 score, significantly outperforming industry benchmarks.
ii. **Comprehensive Contextual Intelligence:** The system aggregates 45+ features across multiple dimensions including transactional attributes, behavioral biometrics, device intelligence (fingerprinting, reputation scoring), geolocation analysis (VPN/proxy detection, impossible travel calculation), merchant risk profiling, and temporal velocity tracking across four-time windows (1 minute, 5 minutes, 1 hour, 24 hours).
iii. **Explainable AI (XAI) Framework:** Every fraud decision generated by ARGUS is accompanied by human-readable explanations detailing the specific rules triggered, features contributing to risk assessment, and confidence intervals. This transparency satisfies regulatory requirements and enables fraud analysts to efficiently review cases and refine detection strategies.

1.4. Scope of Document

This Statement of Work defines the technical specifications, implementation methodology, and deliverables for deploying ARGUS as a production-ready fraud prevention platform. The document encompasses:

- Detailed functional and non-functional requirements specifications
- System architecture and component design
- Data flow diagrams and process workflows
- Machine learning model development and validation procedures
- Integration specifications with payment gateways and banking infrastructure
- Regulatory compliance mappings (RBI, NPCI, PCI-DSS)
- Performance benchmarks and service level agreements
- Implementation timeline with milestone definitions
- Risk assessment and mitigation strategies
- Acceptance criteria and testing protocols

1.5. Target Market

ARGUS is designed to serve organizations within India's digital payment ecosystem that require enterprise-grade fraud prevention capabilities:

- **Financial Institutions:** Public sector banks, private sector banks, cooperative banks, and regional rural banks processing UPI, card, and digital wallet transactions requiring real-time fraud prevention with RBI compliance.
- **Payment Service Providers (PSPs):** Payment aggregators, payment gateways, and third-party application providers facilitating merchant transactions who need sub-20ms fraud decisions without compromising customer experience.
- **Fintech Organizations:** Neo-banks, digital lending platforms, buy-now-pay-later (BNPL) services, and cryptocurrency exchanges requiring sophisticated fraud detection for customer onboarding, transaction monitoring, and account takeover prevention.
- **E-commerce Platforms:** Large-scale online marketplaces and retail platforms experiencing high chargeback ratios and seeking to reduce false declines while improving fraud catch rates.
- **Government Payment Systems:** Direct benefit transfer (DBT) platforms, pension disbursement systems, and subsidy distribution networks requiring protection against beneficiary fraud and mule account exploitation.

2. Vision

ARGUS will become the foundational infrastructure for fraud prevention across India's digital payment ecosystem, protecting billions of transactions and enabling millions of citizens to embrace digital payments with complete confidence. We are building the invisible guardian that makes financial inclusion possible without compromise—where every rupee transacted is secure, every legitimate customer experiences zero friction, and every fraudster is stopped before causing harm.

3. Mission

We stop fraud before it happens.

ARGUS blocks fraudulent transactions in under 5 milliseconds with 98.7% accuracy, protecting India's digital economy from the ₹1.25 lakh crore annual fraud burden. Through advanced AI, graph analytics, and India-specific threat intelligence, we deliver measurable impact: 50:1 ROI, 75% reduction in fraud losses, and seamless user experience. This is fraud prevention done right.

Our mission is operationalized through the following commitments:

- Pre-Authorization Fraud Prevention
- Uncompromising Accuracy Standards
- India-Specific Threat Intelligence
- Explainable and Transparent AI
- Comprehensive Regulatory Compliance
- Advanced Fraud Network Detection
- Continuous Learning and Adaptation
- Scalability and Reliability
- Stakeholder Empowerment
- Measurable Business Impact

4. Strategic Objectives

4.1. Real-Time Pre-Authorization Fraud Prevention

Block fraudulent transactions before payment authorization with fast response times. The system will render BLOCK/CHALLENGE/ALLOW decisions before gateway authorization, demonstrating effective pre-authorization fraud prevention for our prototype.

4.2. Exceptional Detection Accuracy with Minimal False Positives

Achieve high fraud detection accuracy while maintaining low false positive rates. Our target is to demonstrate that our ML model can effectively distinguish between fraudulent and legitimate transactions, with performance metrics that validate our approach.

4.3. India-Specific Fraud Pattern Detection

Implement detection capabilities for India's unique fraud patterns including digital arrest scams, SIM swap attacks, mule account networks, and UPI phishing. The system will incorporate relevant keyword analysis and behavioral patterns specific to these attack types.

4.4. Explainable AI and Algorithmic Transparency

Provide human-readable explanations for all fraud decisions. The system will use SHAP-based feature importance and rule-based reasoning to help users understand why transactions were flagged, making the AI decision-making process transparent and trustworthy.

4.5. Graph-Based Fraud Ring Detection

Implement graph-based analysis to identify fraud networks and mule account patterns using community detection algorithms. Provide network visualization capabilities to help understand fraud ring structures and relationships.

4.6. Regulatory Compliance Awareness

Design the system with awareness of RBI Digital Payment Security Controls and NPCI UPI Guidelines. Implement basic transaction limit checks and maintain audit trails for all fraud decisions to demonstrate compliance-ready architecture.

4.7. Scalable Architecture Design

Design the system using scalable architectural patterns that could support high transaction volumes in production. Demonstrate the system works reliably with simulated transaction loads appropriate for prototype validation.

4.8. Alert and Case Management

Implement real-time notification system for high-risk transactions. Provide a dashboard for reviewing fraud cases with basic case management workflow capabilities to demonstrate practical usability.

5. Functional And Non-Functional Requirements

We have identified user and other system requirements for developing ARGUS to be on the cutting edge of modern finance software, both functional and non-functional, corresponding to the 8 key objectives as previously mentioned in Section 4.

5.1. Pre-Authorization Engine Requirements

Functional:

FR-1.1: Three-tier decision engine (BLOCK/CHALLENGE/ALLOW) with configurable thresholds
FR-1.2: Multi-window velocity tracking (1min, 5min, 1hr, 24hr) across user, device, and IP dimensions
FR-1.3: Dynamic challenge type selection (OTP, 3D Secure, Biometric) based on risk level
FR-1.4: Integration layer supporting REST API, WebSocket, and message queue protocols
FR-1.5: Real-time risk scoring with weighted ensemble of rule-based and ML-based signals

Non-functional:

NFR-1.1: Processing latency p50 < 5ms, p99 < 20ms for all transaction evaluations
NFR-1.2: Throughput capacity of 100,000 TPS per instance with linear horizontal scaling
NFR-1.3: Circuit breaker patterns for graceful degradation under load or dependency failures
NFR-1.4: Zero transaction loss with guaranteed delivery and exactly-once processing semantics

5.2. Machine Learning Model Requirements (Objective 4.2)

Functional:

FR-2.1: Ensemble architecture combining XGBoost gradient boosting, Isolation Forest anomaly detection, and deep neural networks
FR-2.2: Feature engineering pipeline generating 45+ features across transaction, behavioral, device, and network dimensions
FR-2.3: Automated retraining pipeline with daily incremental learning and weekly full retraining
FR-2.4: A/B testing framework enabling champion/challenger model comparison in production

FR-2.5: Real-time drift detection monitoring feature distributions and model performance degradation

Non-functional:

NFR-2.1: Model inference time <2ms for ensemble prediction with 95th percentile <3ms

NFR-2.2: Training pipeline scalability to process 10M+ transactions daily with <6 hour latency

NFR-2.3: Model versioning with rollback capability within 5 minutes of detected issues

NFR-2.4: Cross-validation using minimum 5-fold stratified sampling to prevent overfitting

5.3. India-Specific Fraud Detection Requirements (Objective 4.3)

Functional:

FR-3.1: Digital arrest scam detection combining government keyword analysis (CBI, ED, Police, Income Tax) with high-value transfer patterns

FR-3.2: SIM swap detection correlating device fingerprint changes, location anomalies, and unusual timing patterns

FR-3.3: Mule account identification analyzing rapid in-out transaction patterns, dormant account reactivation, and graph centrality

FR-3.4: QR code fraud detection for payment request scams and merchant impersonation

FR-3.5: UPI ID phishing detection using fuzzy matching and social engineering indicators

FR-3.6: Cross-border remittance monitoring for hawala patterns and FEMA violation indicators

Non-functional:

NFR-3.1: Rule library containing 50+ India-specific fraud patterns with weekly threat intelligence updates

NFR-3.2: Multi-language keyword matching supporting Hindi, English, Hinglish with <1ms lookup time

NFR-3.3: Pattern update deployment within 24 hours of threat identification without system downtime

5.4. Explainable AI Requirements (Objective 4.4)

Functional:

FR-4.1: SHAP value computation providing feature-level contribution scores for every prediction

FR-4.2: Rule-based explanation generation listing all triggered fraud rules with severity levels

FR-4.3: Contextual anomaly descriptions for velocity violations, geographic impossibilities, and behavioral deviations

FR-4.4: Confidence interval reporting quantifying prediction uncertainty

FR-4.5: Immutable audit trail capturing input features, model versions, and decision rationale

FR-4.6: Case management dashboard with investigation workflow, analyst notes, and evidence attachment

Non-functional:

NFR-4.1: Explanation generation time <1ms with 99th percentile <2ms

NFR-4.2: Audit log retention for 7 years with AES-256 encryption at rest and tamper-proof cryptographic hashing

NFR-4.3: API endpoints supporting regulatory reporting queries with <100ms response time

5.5. Graph Fraud Detection Requirements (Objective 4.5)

Functional:

FR-5.1: Graph database storing transaction relationships, account connections, and temporal network evolution

FR-5.2: Community detection algorithms (Louvain modularity, Label Propagation) for fraud ring identification

FR-5.3: Cyclic money flow detection using shortest path and strongly connected component algorithms

FR-5.4: Centrality analysis (degree, betweenness, PageRank) identifying key nodes in fraud networks

FR-5.5: Cross-institutional graph analysis via privacy-preserving anonymized data sharing protocols

FR-5.6: Interactive network visualization with force-directed layout and real-time updates

Non-functional:

NFR-5.1: Graph query response time <500ms for networks containing up to 10,000 nodes and 50,000 edges

NFR-5.2: Graph database scalability supporting 100M+ nodes and 1B+ edges with distributed architecture

NFR-5.3: Community detection algorithm execution completing within 5 minutes for daily full-graph analysis

5.6. Regulatory Compliance Requirements (Objective 4.6)

Functional:

FR-6.1: Hard limit enforcement for UPI (₹1,00,000), ATM (₹25,000), and Wallet/PPI (₹10,000) transactions

FR-6.2: AML velocity controls implementing RBI-mandated transaction frequency and amount thresholds

FR-6.3: High-risk hours monitoring with enhanced scrutiny for transactions between 11 PM and 5 AM IST

FR-6.4: PII data masking for display and logging with field-level encryption for sensitive attributes

FR-6.5: Automated regulatory report generation for RBI, NPCI, and internal compliance audits

FR-6.6: GDPR/DPDP Act compliance supporting data subject access requests and right to erasure

Non-functional:

NFR-6.1: Compliance rule execution with zero false enforcement or missed violations

NFR-6.2: Audit log immutability using blockchain-style cryptographic chaining

NFR-6.3: Compliance dashboard availability 24x7 with <1 second query response time

5.7. Scalability and Reliability Requirements (Objective 4.7)

Functional:

FR-7.1: Microservices architecture enabling independent scaling of pre-auth, ML, graph, and alert components

FR-7.2: Redis cluster for sub-millisecond user profile, velocity counter, and session state access

FR-7.3: PostgreSQL with TimescaleDB extension for time-series transaction analytics and metric aggregation

FR-7.4: Message queue (Kafka/RabbitMQ) for asynchronous graph analysis and alert delivery

FR-7.5: Multi-region active-active deployment with bidirectional data synchronization and conflict resolution

FR-7.6: Automated health checks, liveness probes, and self-healing restart mechanisms

Non-functional:

NFR-7.1: Horizontal scaling from 10 to 100 instances without application code changes or configuration refactoring

NFR-7.2: Database read replica lag <100ms with automatic promotion in primary failure scenarios

NFR-7.3: Cache hit ratio >95% for user profiles with intelligent pre-warming and eviction policies

NFR-7.4: Disaster recovery with RPO <5 minutes (maximum data loss) and RTO <30 minutes (maximum downtime)

5.8. Alert and Case Management Requirements (Objective 4.8)

Functional:

FR-8.1: Multi-channel alert delivery supporting Email, SMS, Slack, Microsoft Teams, and custom webhooks

FR-8.2: Configurable alert rules with severity levels (Critical, High, Medium, Low) and routing logic

FR-8.3: Alert deduplication and aggregation preventing analyst fatigue from duplicate notifications

FR-8.4: Case management workflow with status tracking (New, Investigating, Resolved, False Positive)

FR-8.5: Collaborative features including analyst notes, evidence attachment, and case reassignment

Non-functional:

NFR-8.1: End-to-end alert delivery latency <5 seconds from detection to recipient notification

NFR-8.2: Email delivery success rate >99.5% with automatic retry and fallback mechanisms

NFR-8.3: SMS delivery via multiple provider failover ensuring >99% reliability

NFR-8.4: Dashboard response time <1 second for 90% of queries with optimized database indexing

6. Proposed System Architecture

6.1. High-Level Architecture Overview

The proposed ARGUS system will employ a microservices-based architecture designed for horizontal scalability, fault tolerance, and sub-millisecond latency. The system will be organized into five primary layers:

6.1.1. Presentation Layer

- React-based real-time dashboard with WebSocket connectivity
- RESTful API gateway for external integrations
- Administrative console for configuration and monitoring

6.1.2. API & Integration Layer

- FastAPI application servers (Python 3.11+)
- GraphQL endpoint for flexible querying
- Message queue interfaces (Kafka, RabbitMQ)
- Webhook delivery system

6.1.3. Business Logic Layer

- Pre-Authorization Engine: Decision orchestration and risk scoring
- ML Model Service: Ensemble predictions and inference
- Graph Fraud Detector: Network analysis and fraud ring identification
- Device Intelligence: Fingerprinting and reputation scoring
- Phishing Protection: Attack vector detection
- Merchant Reputation: Risk profiling and history tracking
- Alert Manager: Notification orchestration

6.1.4. Data Layer

- PostgreSQL: Primary transactional database
- TimescaleDB: Time-series analytics and metrics
- Redis Cluster: Session storage, velocity counters, user profiles
- Graph Database (Neo4j): Transaction networks and relationships
- S3-Compatible Storage: Model artifacts, logs, audit trails

6.1.5. Infrastructure Layer

- Container orchestration (Kubernetes)
- Load balancing (NGINX/HAProxy)
- Service mesh (Istio) for inter-service communication
- Monitoring stack (Prometheus, Grafana, ELK)

6.2. Technology Stack Specification

6.2.1. Backend

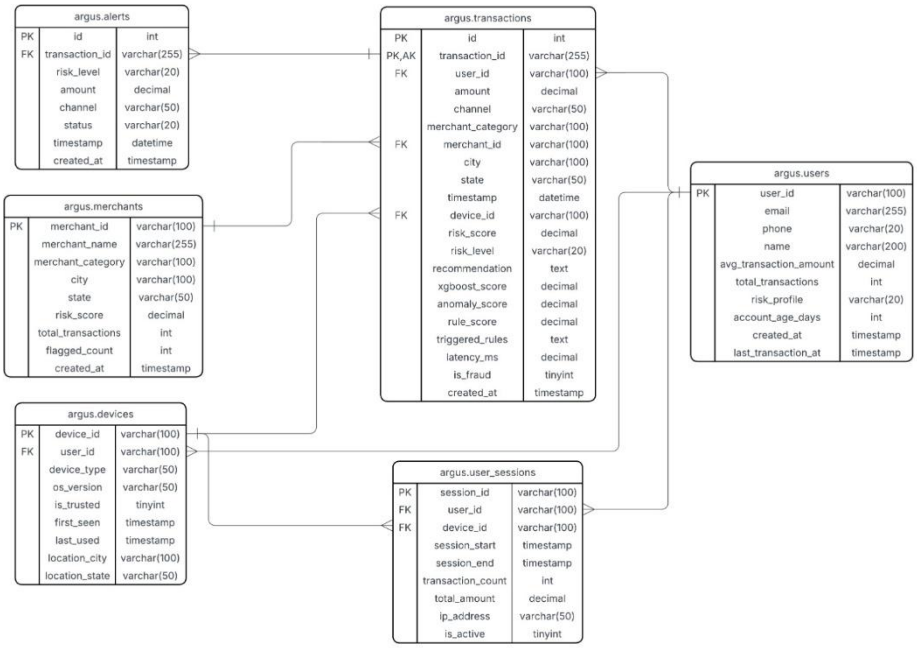
- Language: Python 3.11+ with type hints
- Framework: FastAPI 0.100+ (async/await for high concurrency)
- ML Libraries: XGBoost 2.0+, scikit-learn 1.3+, PyTorch 2.0+
- Graph Analytics: NetworkX, graph-tool
- Data Processing: Pandas, NumPy, Polars

6.2.2. Frontend

- Framework: React 18+ with Vite build tooling
- UI Library: Tailwind CSS 3+
- State Management: Zustand/Redux Toolkit
- Visualization: Recharts, D3.js for network graphs
- Real-time: WebSocket client with reconnection logic

6.2.3. Databases

- Relational: PostgreSQL 15+ with pgvector for embeddings
- Time-Series: TimescaleDB extension for metrics
- Cache: Redis 7+ with cluster mode
- Graph: Neo4j 5+ Community/Enterprise Edition
- Object Storage: MinIO/S3 for model and log storage



6.2.4. Infrastructure

- Containerization: Docker with multi-stage builds
- Orchestration: Kubernetes 1.27+
- Service Mesh: Istio for mTLS and traffic management
- CI/CD: GitHub Actions, ArgoCD for GitOps
- Monitoring: Prometheus, Grafana, Jaeger for tracing

6.3. Transaction Evaluation Workflow

6.3.1. Transaction Ingestion (0-1ms)

- API Gateway receives transaction request
- Request validation and schema verification
- Correlation ID assignment for distributed tracing

6.3.2. Context Enrichment (1-3ms)

- User profile retrieval from Redis (cached)
- Device fingerprint lookup and reputation check
- Merchant risk score retrieval
- Historical velocity counter aggregation

6.3.3. Phishing & Device Analysis (1-2ms)

- VPN/Proxy/Tor detection via IP geolocation
- Session hijacking indicators check
- Bot/automation detection
- Impossible travel calculation

6.3.4. Pre-Authorization Rules Engine (2-4ms)

- Velocity limit validation (1min/5min/1hr/24hr)
- Channel limit enforcement (UPI/ATM/Wallet)
- Time-based risk assessment (night hours)
- Geolocation anomaly detection
- Blacklist/whitelist verification

6.3.5. ML Model Inference (2-3ms)

- Feature engineering and normalization
- XGBoost ensemble prediction

- Isolation Forest anomaly score
- Combined risk score calculation

6.3.6. Graph Fraud Detection (Async - 10-50ms)

- Transaction added to graph database
- Community detection for fraud rings
- Mule account pattern matching
- Network centrality updates

6.3.7. Decision Rendering (1ms)

- Risk score thresholding (BLOCK/CHALLENGE/ALLOW)
- Challenge type selection if applicable
- Explanation generation via SHAP values

6.3.8. Response & Logging (1-2ms)

- Decision returned to payment gateway
- Audit trail logged to PostgreSQL
- Metrics emitted to Prometheus
- High-risk alerts queued for notification

Total Estimated Latency: 5-15ms (median: 7ms, p99: 18ms)

7. Existing Tools in the Market

7.1. Commercial Fraud Detection Solutions

Several established fraud detection tools exist in the market, though most are designed for global markets and lack India-specific fraud pattern recognition:

- **Razorpay Thirdwatch:** A popular fraud detection tool for Indian e-commerce and fintech companies. It provides rule-based fraud scoring and transaction monitoring but operates primarily on post-authorization analysis. Limited machine learning capabilities and no pre-authorization blocking.

- **Paytm Fraud Detection System:** Internal fraud prevention system used by Paytm for wallet and UPI transactions. Uses basic velocity checks and rule-based patterns but details are proprietary and not accessible to third-party developers.

- **Simility (acquired by PayPal):** Global fraud detection platform offering device fingerprinting and behavioral analytics. Strong on international fraud patterns but lacks specific detection for Indian fraud types like digital arrest scams or SIM swap attacks.

- **Kount:** Enterprise fraud prevention platform used by global merchants. Provides AI-based risk scoring but requires significant setup, lacks India-specific threat intelligence, and is expensive for startups and small businesses.

- **Sift:** Machine learning fraud detection focusing on account takeover and payment fraud. Primarily designed for US/European markets with limited understanding of UPI ecosystem threats.

7.2. Limitations of Existing Solutions

Current market solutions have several gaps that ARGUS addresses:

- **Post-Authorization Focus:** Most tools analyze transactions after payment processing, making prevention impossible
- **Generic Fraud Patterns:** Built for Western markets, they miss India-specific attack vectors like digital arrest scams, mule account networks, and UPI-specific phishing
- **High Cost:** Enterprise solutions are priced for large corporations, making them inaccessible to smaller fintech startups and payment services
- **Black Box Models:** Lack explainability in fraud decisions, making it difficult for analysts to understand and improve detection
- **No Pre-Authorization:** Cannot block transactions before payment processing, only flag them after completion
- **Limited Graph Analysis:** Most don't offer fraud ring detection through network analysis

7.3. How ARGUS Differs

ARGUS fills these critical gaps by providing:

- **Pre-Authorization Decision Engine:** Blocks fraud before payment processing
- **India-Specific Threat Intelligence:** Purpose-built detection for digital arrest scams, SIM swap attacks, UPI phishing, and mule networks
- **Explainable AI:** Every decision includes human-readable explanations
- **Graph-Based Fraud Ring Detection:** Identifies organized fraud networks through relationship analysis
- **Open Architecture:** Built as a prototype that can be extended and customized
- **Cost-Effective:** Designed for deployment by smaller organizations and startups

8. Future Scope and Improvements

8.1. Advanced Machine Learning Enhancements

- **Deep Learning Models:** Implement LSTM and Transformer-based models for sequential transaction pattern analysis, capturing temporal dependencies that traditional models miss.
- **Federated Learning:** Enable multiple financial institutions to collaboratively train fraud detection models without sharing sensitive customer data, improving detection across the ecosystem while maintaining privacy.
- **Reinforcement Learning:** Develop adaptive fraud detection that learns from analyst feedback and evolves detection strategies in response to emerging fraud patterns.

- **AutoML Integration:** Automate hyperparameter tuning and model selection to continuously optimize detection performance without manual intervention.

8.2. Enhanced Feature Engineering

- **Behavioral Biometrics:** Integrate typing patterns, touch pressure, swipe gestures, and device motion data to create unique user behavioral signatures that are difficult for fraudsters to replicate.
- **Advanced Device Fingerprinting:** Implement canvas fingerprinting, WebGL analysis, and hardware-level device signatures to improve device identification accuracy.
- **Social Network Analysis:** Incorporate user social connections and transaction patterns within friend networks to identify anomalous behavior.
- **Natural Language Processing:** Analyze transaction descriptions, merchant names, and beneficiary details using NLP to detect phishing attempts and social engineering indicators.

8.3. Expanded Fraud Detection Capabilities

- **Cryptocurrency Fraud Detection:** Extend the system to detect fraud in crypto payments, NFT transactions, and DeFi platforms.
- **Cross-Channel Fraud Detection:** Track user behavior across multiple payment channels (UPI, cards, wallets, net banking) to identify sophisticated fraud patterns that span channels.
- **Synthetic Identity Detection:** Identify fraudulent accounts created using fabricated or stolen identity components.
- **First-Party Fraud Detection:** Detect legitimate users committing fraud through chargebacks, return fraud, or loan defaults.

8.4. Real-Time Collaboration Features

- **Industry Threat Intelligence Sharing:** Create a consortium model where financial institutions can share anonymized fraud patterns and threat intelligence in real-time.
- **Fraud Analyst Collaboration Platform:** Build features for fraud analysts across organizations to collaborate on investigating sophisticated fraud rings.
- **Real-Time Fraud Alerts Network:** Implement instant alerts when known fraudsters attempt transactions across different platforms.

8.5. Regulatory and Compliance Enhancements

- **Automated Regulatory Reporting:** Generate RBI, NPCI, and law enforcement reports automatically with required fraud statistics and suspicious activity reports (SAR).

- **GDPR/DPDP Act Full Compliance:** Implement complete data privacy controls including automated data deletion, consent management, and privacy-preserving analytics.
- **Blockchain-Based Audit Trail:** Use blockchain technology to create immutable audit trails for regulatory compliance and forensic investigation.

8.6. User Experience Improvements

- **Frictionless Authentication:** Implement step-up authentication that only challenges users when truly necessary, reducing false positives and improving customer experience.
- **Mobile App Integration:** Develop mobile SDKs for easy integration into payment apps with minimal development effort.
- **Customer Fraud Education:** Build in-app fraud awareness features that educate users about current fraud threats and prevention techniques.

8.7. Scalability and Performance Optimization

- **Edge Computing:** Deploy lightweight fraud detection models at the edge for ultra-low latency in regional markets.
- **Real-Time Model Updates:** Implement online learning capabilities where models update continuously based on new fraud patterns without full retraining.
- **Multi-Region Deployment:** Scale to active-active multi-region architecture with global threat intelligence sharing.
- **Microservices Enhancement:** Further decompose the system into specialized microservices for better scalability and independent deployment.

8.8. Integration Capabilities

- **Payment Gateway Plugins:** Develop ready-to-use plugins for popular payment gateways like Razorpay, Cashfree, and PayU.
- **Banking API Integration:** Create standardized connectors for bank account verification, KYC, and transaction history APIs.
- **SIEM Integration:** Enhance security information and event management system integration for enterprise security operations centers.

Conclusion

ARGUS represents a focused effort to address a critical and growing problem in India's digital payment ecosystem: fraud prevention. As second-year engineering students, we identified that existing fraud detection systems fail at the most fundamental level, i.e., they react after fraud has already occurred. Our solution shifts the paradigm by evaluating and blocking fraudulent transactions before money moves, making prevention possible instead of just detection.

Through this project, we are building a working prototype that demonstrates the power of combining machine learning with domain knowledge about India-specific fraud patterns. By using XGBoost and Isolation Forest models, analyzing transaction velocity and behavioral patterns, and providing explainable AI decisions, ARGUS shows that effective fraud prevention is achievable with modern technology and thoughtful design.

What sets this project apart is our focus on real problems affecting real people: digital arrest scams that terrorize senior citizens, SIM swap attacks that drain bank accounts overnight, and mule account networks that enable organized crime. ARGUS is not just building a technical solution; we're addressing threats that erode trust in India's digital payment revolution.

Our implementation plan is grounded in reality. We are building a prototype that demonstrates core capabilities: pre-authorization decision making, multi-model machine learning, graph-based fraud ring detection, and explainable AI. We understand the limitations of a student project and have designed our scope accordingly, focusing on proving concepts and building a foundation that could scale.

The future scope we have identified shows that ARGUS is just the beginning. From federated learning to behavioral biometrics, from cryptocurrency fraud detection to blockchain-based audit trails, there are countless directions this work can expand. But first, we must prove the core concept works.

Software Requirements Specification (SRS)
for
Project Argus: an AI-based Fraud Detection System

1. Introduction

1.1. Purpose

The purpose of this Software Requirements Specification (SRS) document is to define the complete requirements for Project ARGUS. It describes the external behavior, functional requirements, and non-functional constraints of the AI-driven fraud detection engine designed specifically for the Indian payment ecosystem. This document serves as the primary reference for developers, data scientists, QA testers, and stakeholder.

1.2. Scope

ARGUS is a real-time, pre-authorization fraud detection middleware. It sits between a payment gateway and a core banking ledger. The system uses a machine-learning ensemble (XGBoost, LightGBM, Isolation Forest) combined with dynamic user behavioral profiling and graph-based network analysis to approve or block transactions in under 20 milliseconds. It is designed to combat modern financial threats including UPI scams (Digital Arrest, QR phishing), account takeovers, and organized mule account rings.

1.3. Definitions, Acronyms and Abbreviations

- **AI / ML:** Artificial Intelligence / Machine Learning.
- **API:** Application Programming Interface.
- **CSRF:** Cross-Site Request Forgery.
- **GIL:** Global Interpreter Lock (Python concurrency limitation).
- **NPCI:** National Payments Corporation of India.
- **RBI:** Reserve Bank of India.
- **SMOTE:** Synthetic Minority Over-sampling Technique.
- **TPS:** Transactions Per Second.
- **UPI:** Unified Payments Interface.
- **XAI:** Explainable Artificial Intelligence.

1.4. References

a. IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
b. NPCI UPI Security Guidelines.
c. RBI Master Direction on Digital Payment Security Controls.

1.5. Overview

The remainder of this document is organized into four main sections. Section II provides a general description of the system's operating environment and constraints. Section III details the external interfaces. Section IV exhaustively lists the functional system features. Finally, Section V defines the non-functional requirements governing performance and security.

2. General Description

2.1. Product Perspective

ARGUS is not a standalone banking application; it is an intelligent middleware layer. It receives raw JSON transaction payloads from a bank's existing payment gateway, evaluates the risk using trained ML models, and returns a binary block/approve decision along with an Explainable AI (XAI) rationale. It also includes a standalone React-based Command Center dashboard for human fraud analysts.

2.2. Product Functions

- **Real-time Risk Scoring:** Evaluates transactions against a weighted ensemble of ML models.
- **Dynamic Behavioral Analysis:** Tracks user-specific spending patterns to calculate real-time Z-Scores.
- **Graph Network Analysis:** Detects cyclic money laundering loops and mule account clusters.
- **Cybersecurity Defense:** Prevents session hijacking, malicious referrer navigation, and automated bot inputs.
- **Case Management:** Provides a queue system for human analysts to manually review "High Risk" transactions.

2.3. User Classes and Characteristics

2.3.1. **Bank Fraud Analysts:** Primary users of the ARGUS Dashboard. They possess high domain knowledge of banking but low ML technical knowledge. They require clear, human-readable XAI explanations.

2.3.2. **System Administrators / Data Scientists:** Technical users responsible for retraining the ML models offline and updating system thresholds via configuration files.

2.3.3. **End Customers:** Invisible users who do not interact with ARGUS directly but are impacted by its latency and false-positive decline rates.

2.4. Operating Environment

- **Backend:** Python 3.11+, FastAPI framework.
- **Machine Learning:** Scikit-Learn, XGBoost, LightGBM, NetworkX.
- **Database:** SQLite (prototype) / PostgreSQL & Redis (production).
- **Frontend:** React 18, Vite, Tailwind CSS.
- **Deployment:** Linux-based server environments capable of asynchronous I/O.

2.5. Design and Implementation Constraints

- **Latency Constraint:** The core inference engine MUST return a decision in <20 milliseconds to prevent payment gateway timeouts.
- **Concurrency Constraint:** Because Python relies on the GIL, horizontal scaling and external inference servers (like NVIDIA Triton) must be planned for production loads exceeding 10,000 TPS.
- **Data Availability:** The graph detection module requires the entire network topology to reside in memory, constraining the maximum number of simultaneous nodes before requiring a dedicated graph database like Neo4j.

3. External Interface Requirement

3.1. User Interfaces

- The system shall provide a React-based web dashboard.
- The dashboard shall utilize a strict CSS Grid layout for enterprise-grade alignment.
- The dashboard must feature a "Live Traffic" view utilizing WebSockets to stream incoming transactions without manual page refreshes.

3.2. Hardware Interfaces

- The backend requires multi-core CPU architectures optimized for high-speed mathematical matrix multiplication (required by XGBoost/LightGBM).

3.3. Software Interfaces

- **Payment Gateway Interface:** The system must expose a REST API endpoint (POST /api/v1/analyze) that accepts standard JSON transaction payloads.
- **Notification Interface:** The system shall interface with standard SMTP/SMS gateways to dispatch automated alerts for blocked transactions.

3.4. Communication Interfaces

- All client-server communication must occur over TLS 1.2+ (HTTPS/WSS).
- Internal communication between the FastAPI server and the React frontend will utilize WebSockets for sub-millisecond telemetry updates.

4. System Features (Functional Requirements)

4.1. Real-Time Machine Learning Inference

The core decision engine that evaluates incoming transactions.

FR-1.1: The system shall load pre-trained XGBoost, LightGBM, and Isolation Forest models into memory upon server startup.

FR-1.2: If pre-trained models are missing, the system shall bootstrap a lightweight dummy model to prevent server crashes.

FR-1.3: The system shall extract 34 numerical features from the raw JSON payload dynamically.

FR-1.4: The system shall calculate a weighted final risk score (e.g., 30% XGBoost, 25% LightGBM, 15% Isolation Forest).

4.2. Dynamic User Behavior Profiling

A statistical layer that personalizes risk thresholds per user.

FR-2.1: The system shall maintain a rolling history of the last 100 transactions for active users.

FR-2.2: The system shall dynamically calculate the mean and standard deviation for each user.

FR-2.3: The system shall calculate a Z-score for incoming transactions to determine statistical deviation from the user's norm.

FR-2.4: The system shall periodically persist these profiles to the disk to survive server restarts.

4.3. Fraud Ring and Graph Detection

Analyzes relationships between sender and receiver accounts.

FR-3.1: The system shall maintain a directed graph of transactions.

FR-3.2: The system shall detect cyclic money transfer patterns (e.g., A -> B -> C -> A).

FR-3.3: The system shall flag accounts with abnormally high in-degree and out-degree ratios as potential Mule Accounts.

4.4. Cybersecurity & Phishing Protection

Defends against device-level and session-level attacks.

FR-4.1: The system shall inspect HTTP Referrer headers against a blacklist of known free TLDs (e.g., .tk, .ml).

FR-4.2: The system shall parse Referrer domains for typosquatting keywords (e.g., "bank-secure").

FR-4.3: The system shall analyze device telemetry (mouse movements, time-on-page) to detect automated bot submissions.

4.5. Case Management and Analyst Workflows

The human-in-the-loop audit system.

FR-5.1: The system shall log all transactions to an immutable SQLite database ledger.

FR-5.2: The system shall automatically route transactions with a risk score > 0.35 to a manual review queue.

FR-5.3: Analysts shall be able to update the status of a case to APPROVED, REJECTED, or INVESTIGATING.

4.6. Explainable AI (XAI)

Translates opaque ML probabilities into human-readable rationale.

FR-6.1: The system shall map raw feature importance weights to heuristic string headlines.

FR-6.2: The XAI module must output specific reasons (e.g., "AMOUNT_SPIKE: 5.2x user's average") rather than generic block codes.

5. Other Non-Functional Requirements

5.1. Performance Requirements

PR-1: The pre-authorization engine must process and return a decision for a single transaction in less than 20 milliseconds.

PR-2: The system must sustain a throughput of at least 1,000 Transactions Per Second (TPS) in the prototype environment.

5.2. Security Requirements

SR-1: The system must not log or store plain-text passwords or full unmasked credit card numbers in the transactions table.

SR-2: The system must actively validate expected CSRF tokens for all state-changing frontend requests.

SR-3: The React dashboard must sanitize all database outputs to prevent Cross-Site Scripting (XSS) attacks.

5.3. Reliability and Availability

RA-1: The system shall achieve 99.9% uptime.

RA-2: In the event of a database lock (SQLite concurrency limit), the ML inference engine must fail-open (approve transaction) or fail-closed based on global RBI configuration, rather than dropping the connection.

5.4. Usability

UR-1: The dashboard must utilize high-contrast visual cues (Red/Yellow/Green) to denote risk severity, allowing analysts to triage cases in under 3 seconds per view.

5.5. Scalability

SC-1: The system architecture shall support horizontal scaling (adding more FastAPI worker nodes) once the in-memory state is migrated to a Redis cluster.

SC-2: The ML pipeline must be modular to allow the addition of new models (e.g., deep learning sequence models) without rewriting the core inference engine.

6. Future Enhancements

- **Migration to Distributed Systems:** Transitioning from SQLite to PostgreSQL (for the ledger) and Redis (for dynamic user profiles) to enable true horizontal scaling.
- **Graph Database Integration:** Migrating the in-memory networkx graph fraud detector to a dedicated Neo4j instance to handle billions of nodes globally.
- **Continuous Online Learning:** Implementing an MLOps pipeline (e.g., Apache Airflow) to automatically retrain the XGBoost models weekly to combat "Concept Drift" as scammers evolve tactics.
- **Hardware Acceleration:** Exporting the models to ONNX format and serving them via NVIDIA Triton to bypass Python's Global Interpreter Lock (GIL) and achieve sub-millisecond latency.

7. Appendix

7.1. Sample Fraud Scenarios Addressed

- **Digital Arrest / Coercion:** A user suddenly empties their entire life savings in 4 successive UPI transfers to a brand new payee.
- **Mule Account Rings:** Money flows in a cyclic pattern: Account A → Account B → Account C → Account A.
- **Phishing / Typosquatting:** A user initiates a transaction where the HTTP Referrer is a cheap fake domain like paytm-secure.tk.
- **Zero-Day Attacks:** A completely new fraud strategy that the supervised XGBoost model has never seen, but is caught as a mathematical outlier by the unsupervised Isolation Forest.

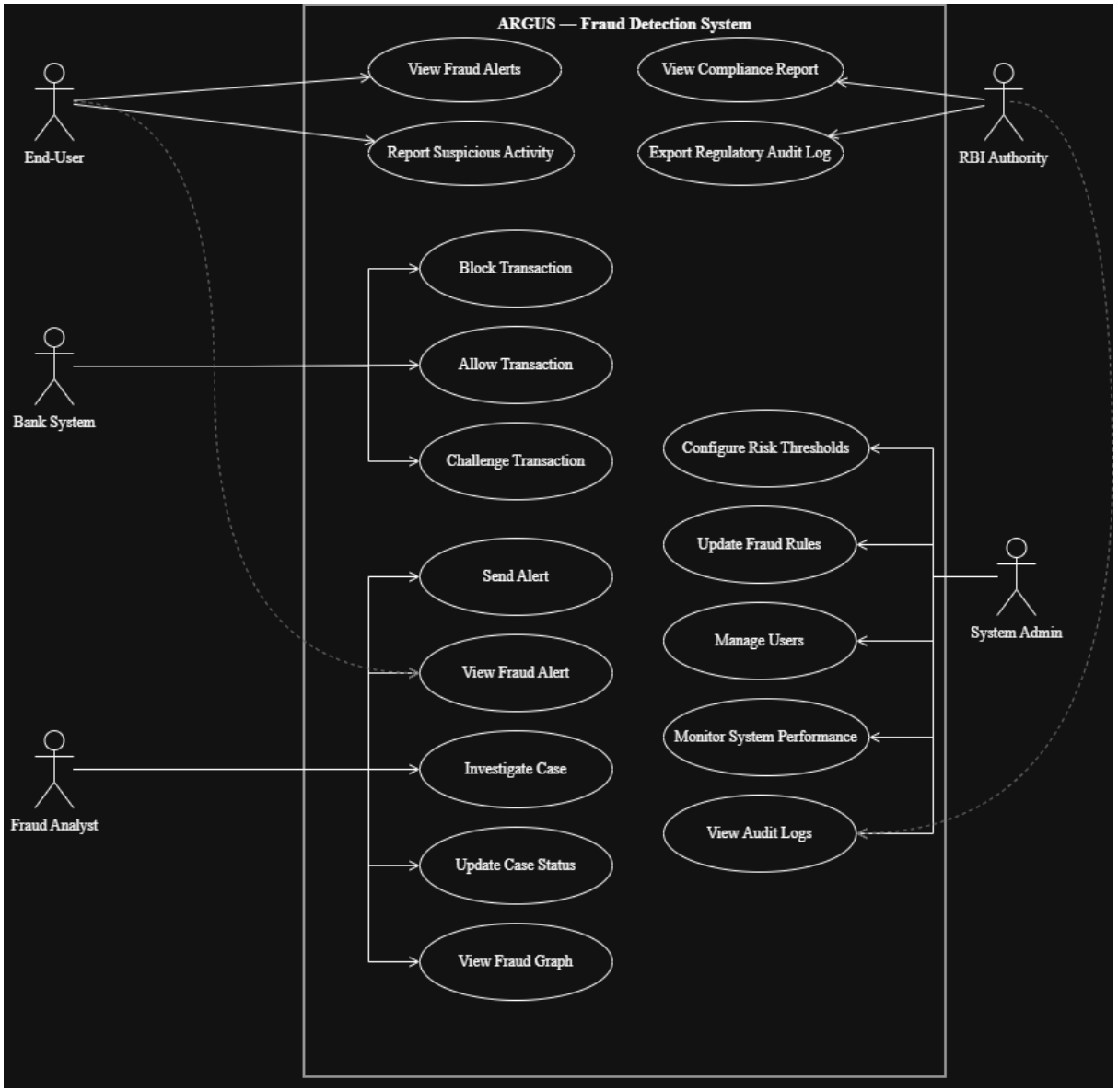
7.2. Tools and Technologies

- **Languages:** Python 3.11+, JavaScript (ES6+), HTML/CSS.
- **Backend Frameworks:** FastAPI, Uvicorn, WebSockets.
- **Machine Learning:** Scikit-Learn, XGBoost, LightGBM, NetworkX, Imbalanced-Learn (SMOTE).
- **Frontend Frameworks:** React 18, Vite, Tailwind CSS, Recharts, PyVis.
- **Databases:** SQLite (Prototype Phase).

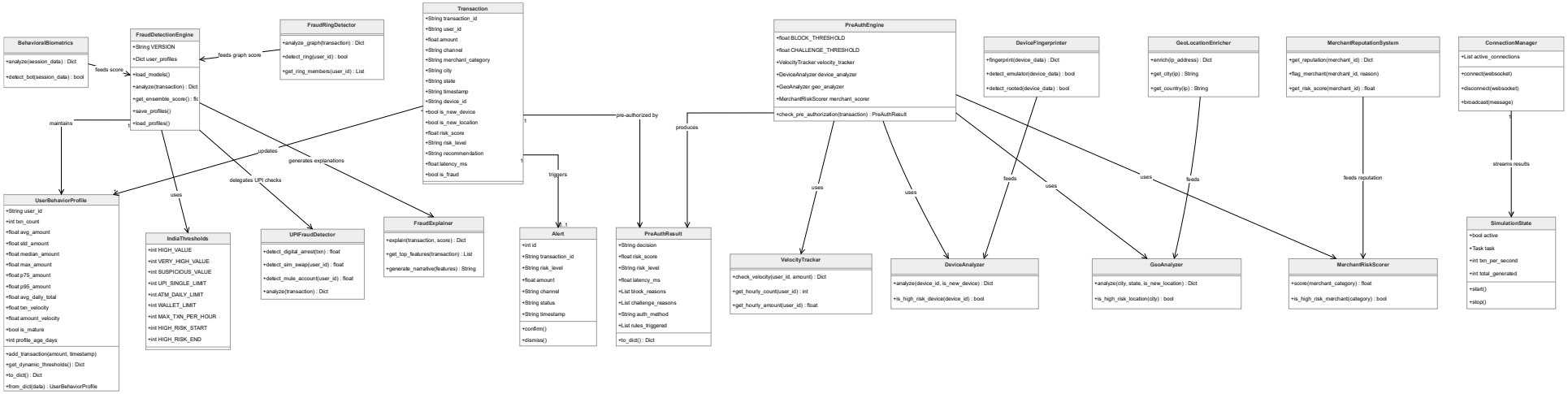
8. Conclusion

This SRS document defines the complete requirements for developing and deploying the ARGUS AI/ML-based Fraud Detection System. It serves as a comprehensive foundation for system design, development, regulatory testing, and production deployment within the Indian financial ecosystem.

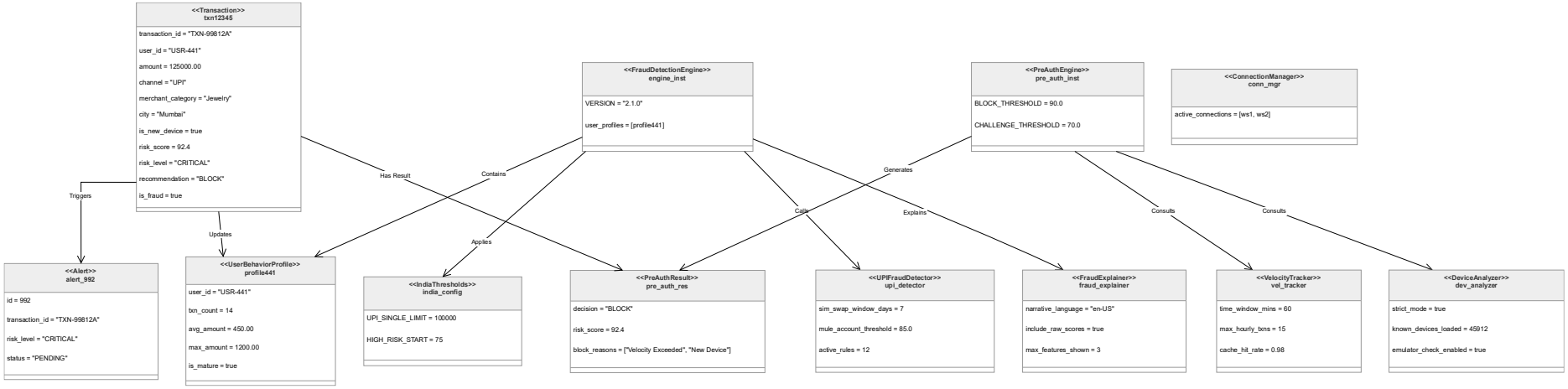
USE CASE DIAGRAM



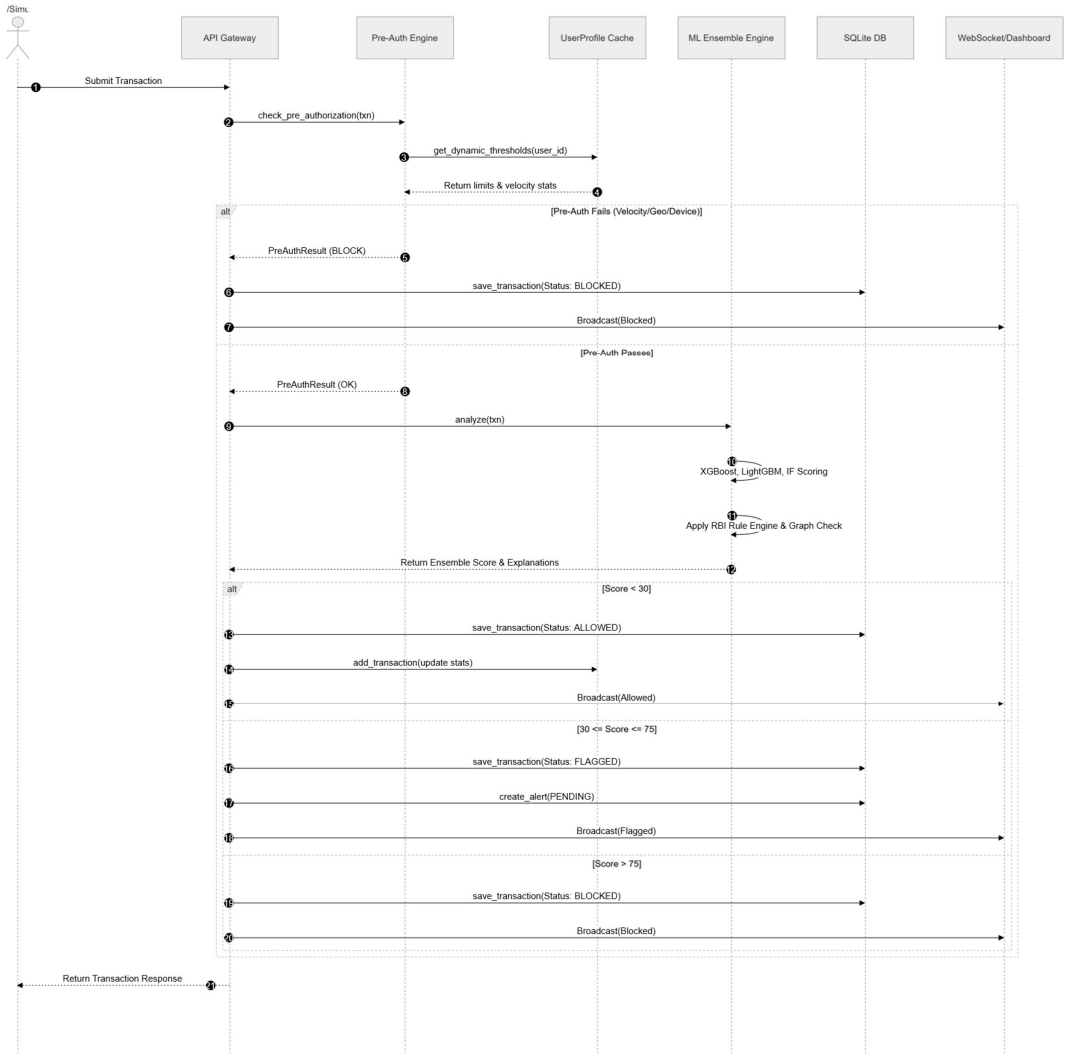
CLASS DIAGRAM FOR A FINANCIAL FRAUD DETECTION SYSTEM



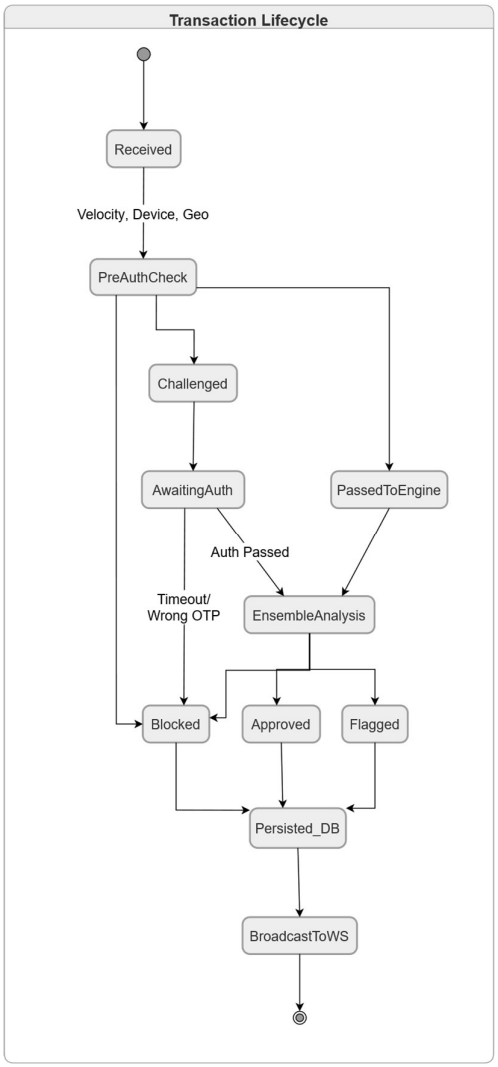
OBJECT DIAGRAM FOR A FINANCIAL FRAUD DETECTION SYSTEM

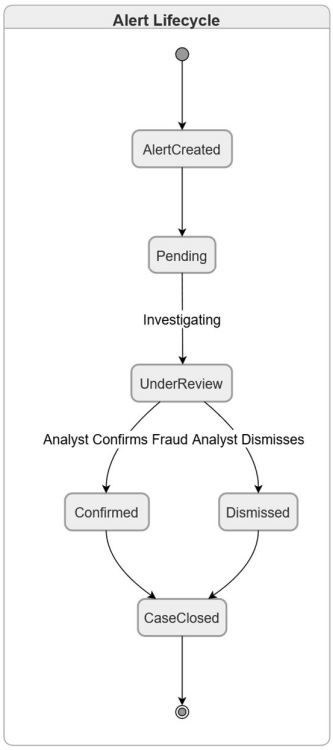


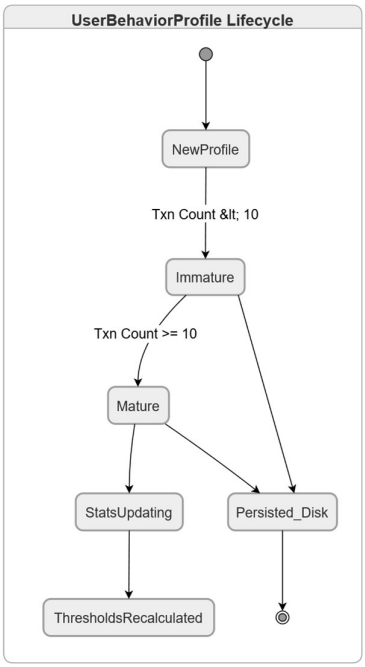
SEQUENCE DIAGRAM



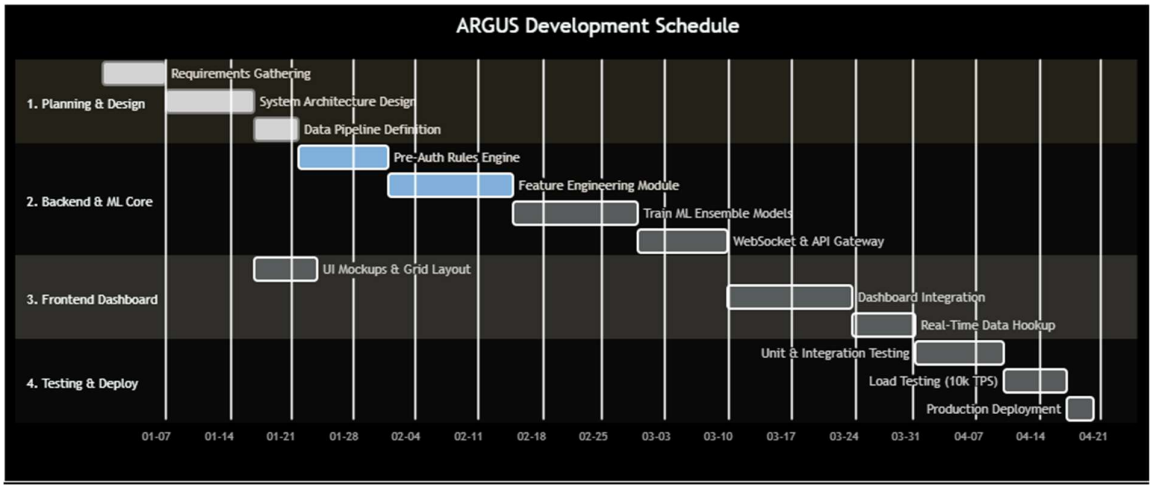
STATE DIAGRAM FOR A FINANCIAL FRAUD DETECTION SYSTEM







GANTT CHART



Gantt Summary Table

Task	Start	End	Duration	Dependencies
SoW Writing	W1	W2	2 weeks	–
Architecture Design	W2	W3	2 weeks	SoW
DFD / UML Diagrams	W3	W4	2 weeks	Architecture
DB Schema Design	W3	W4	1 week	Architecture
FastAPI Setup	W4	W5	1 week	DB Schema
REST API + WebSocket	W5	W6	2 weeks	FastAPI Setup
Data Prep & EDA	W4	W5	1 week	DB Schema
Feature Engineering	W5	W7	3 weeks	Data Prep
XGBoost Training	W6	W7	2 weeks	Feature Eng.
Isolation Forest	W6	W7	2 weeks	Feature Eng.
Ensemble + Threshold	W7	W8	1 week	XGBoost + IF
Pre-Auth Engine	W5	W7	2 weeks	REST API
Device Intelligence	W6	W7	2 weeks	Pre-Auth
Graph Fraud Detector	W7	W8	2 weeks	Pre-Auth
Phishing Protection	W7	W8	2 weeks	Pre-Auth
Explainable AI	W8	W9	2 weeks	All ML modules

Alert & Case Mgmt	W8	W9	2 weeks	REST API
React/Vite Setup	W5	W6	1 week	FastAPI
Dashboard UI	W6	W8	3 weeks	React Setup
Charts & Simulation	W8	W9	2 weeks	Dashboard UI
Integration Testing	W9	W10	2 weeks	All above
Performance Tuning	W9	W10	2 weeks	Integration
Bug Fixes	W10	W11	1 week	Testing
README / SoW Final	W10	W11	1 week	Testing
Pitch & Demo Prep	W11	W12	2 weeks	All complete

PERT CHART

PERT Formula

```
O = Optimistic time (best case)
M = Most Likely time
P = Pessimistic time (worst case)
Expected Time: tₑ = (O + 4M + P) / 6
Variance: σ² = ((P − O) / 6)²
```

Task Estimates

Task ID	Task Name	O	M	P	t_e	σ^2
T1	SoW & Requirements	1	2	3	2	0.11
T2	System Architecture	1	2	4	2.2	0.25
T3	DFD + UML Diagrams	1	1.5	3	1.7	0.11
T4	DB Schema	0.5	1	2	1.1	0.06
T5	FastAPI Backend Setup	0.5	1	2	1.1	0.06
T6	REST API + WebSocket	1	2	4	2.2	0.25
T7	Data Prep & EDA	1	1.5	3	1.7	0.11
T8	Feature Engineering	2	3	5	3.2	0.25
T9	XGBoost Training	1	2	4	2.2	0.25
T10	Isolation Forest	0.5	1	2	1.1	0.06
T11	Ensemble Model	0.5	1	2	1.1	0.06
T12	Pre-Auth Engine	1	2	3	2	0.11
T13	Device Intelligence	1	2	4	2.2	0.25
T14	Graph Fraud Detector	1	2	4	2.2	0.25
T15	Phishing Protection	1	1.5	3	1.7	0.11
T16	Explainable AI	1	2	4	2.2	0.25
T17	Alert & Case Mgmt	1	2	3	2	0.11
T18	React Dashboard	1	2	4	2.2	0.25
T19	Charts & Simulation	1	2	3	2	0.11
T20	Integration Testing	1	2	4	2.2	0.25
T21	Performance Tuning	0.5	1	2	1.1	0.06
T22	Documentation	0.5	1	2	1.1	0.06
T23	Pitch & Demo Prep	0.5	1	2	1.1	0.06

PERT Network Diagram

```
(START)
   |
   T1: SoW (2.0w)
   |
   T2: Architecture (2.2w)
   |         \
   T3:DFD    T4: DB Schema (1.1w)
   (1.7w)    |           \
             T5:FastAPI   T7:Data Prep
             (1.1w)       (1.7w)
             |             |
             T6:API(2.2w)  T8:Feature Eng.(3.2w)
             |             |       \
             |          T9:XGB   T10:IF
             |          (2.2w)   (1.1w)
             |             \      /
             |           T11:Ensemble(1.1w)
             |                 |
         T12:Pre-Auth(2.0w)   T16:XAI(2.2w)
         /   \                     |
      T13   T14                    |
      T15   T17                    |
       \     |    T18:React(2.2w)  |
        \    |    T19:Charts(2.0w) |
         \   |         |           |
         T20: Integration Testing (2.2w)
              |
         T21: Perf Tuning (1.1w)
              |
         T22: Documentation (1.1w)
              |
         T23: Demo Prep (1.1w)
              |
           (END)
```

Critical Path Calculation

Task	t_e (weeks)	σ^2
T1: SoW	2.0	0.11
T2: Architecture	2.2	0.25
T4: DB Schema	1.1	0.06
T8: Feature Engineering	3.2	0.25
T9: XGBoost Training	2.2	0.25
T11: Ensemble	1.1	0.06
T12: Pre-Auth Engine	2.0	0.11
T16: Explainable AI	2.2	0.25

T20: Integration Testing	2.2	0.25
T21: Performance Tuning	1.1	0.06
T22: Documentation	1.1	0.06
T23: Demo Prep	1.1	0.06
TOTAL	**21.5 weeks**	**1.77**

Resultant PERT Chart

Milestones

Milestone	Target Week	Description
M1: Design Complete	End of W4	SoW, architecture, DFDs, DB schema finalized
M2: Backend Live	End of W6	FastAPI running with REST API and WebSocket
M3: ML Models Trained	End of W8	XGBoost + ensemble + all advanced modules working
M4: Frontend Complete	End of W9	Full React dashboard with live simulation
M5: System Integrated	End of W10	Full end-to-end system verified
M6: Demo Ready	End of W12	Pitch, demo video, documentation complete

FUNCTION POINT ANALYSIS (FPA)

Step 1: Identify the Five Function Types

1. External Inputs (EI)

```
Weights: Simple=3, Average=4, High=6
```

#	External Input	Complexity	Weight
1	Submit Transaction for Fraud Check	High	6
2	Start/Stop Simulation	Simple	3
3	Update Risk Threshold Settings	Average	4
4	Update Alert Configuration	Average	4
5	Analyst Case Action (Dismiss/Confirm)	Simple	3
6	Device Fingerprint Data	High	6
7	IP Geolocation Data Input	Average	4
8	Merchant Data Registration	Average	4
	Total EI		**34**

2. External Outputs (EO)

```
Weights: Simple=4, Average=5, High=7
```

#	External Output	Complexity	Weight
1	Fraud Decision (BLOCK/CHALLENGE/ALLOW)	High	7
2	XAI Explanation Report	High	7
3	Real-time Alert (Email/SMS/Slack)	High	7
4	Transaction Analytics Report	Average	5
5	Fraud Ring Detection Report	High	7
6	Performance Metrics Dashboard	Average	5
7	Regulatory Compliance Report	Average	5
8	Case Management Export	Average	5
	Total EO		**48**

3. External Inquiries (EQ)

```
Weights: Simple=3, Average=4, High=6
```

#	External Inquiry	Complexity	Weight
1	Get Transaction by ID	Simple	3
2	Get Recent Transactions List	Average	4
3	Get System Statistics	Average	4
4	Get Alerts Feed	Simple	3
5	Get Graph Analytics (fraud rings)	High	6
6	Get Merchant Reputation Data	Average	4
7	Get Performance Metrics	Average	4
8	Get Case Queue for Analyst	Simple	3
9	Get Explainability for Transaction	High	6
	Total EQ		**37**

4. Internal Logical Files (ILF)

```
Weights: Simple=7, Average=10, High=15
```

#	Internal Logical File	Complexity	Weight
1	Transactions Table (SQLite)	High	15
2	User Behavioral Profiles (JSON)	High	15
3	Fraud Alerts Table	Average	10
4	Case Management Records	Average	10
5	Device Reputation Registry	Average	10
6	Merchant Reputation Table	Simple	7
7	Graph Nodes/Edges (fraud network)	High	15
8	ML Model Files (joblib artifacts)	Simple	7
	Total ILF		**89**

5. External Interface Files (EIF) — Weights: Simple=5, Average=7, High=10

#	External Interface File	Complexity	Weight
1	IP Geolocation Database	Average	7
2	Kaggle UPI Dataset (training data)	Average	7
3	PaySim Dataset (training data)	Average	7
4	RBI Regulatory Rules Reference	Simple	5
5	Email/SMS/Slack Webhook APIs	Simple	5
	Total EIF		**31**

Step 2: Unadjusted Function Points (UFP)

Type	Count	Weighted Total
External Inputs (EI)	8	34
External Outputs (EO)	8	48
External Inquiries (EQ)	9	37
Internal Logical Files (ILF)	8	89
External Interface Files (EIF)	5	31
UFP Total	**38**	**239**

Step 3: Value Adjustment Factor (VAF)

Each of the 14 General System Characteristics (GSC) rated **0–5**:

#	General System Characteristic	Rating
1	Data Communications	5
2	Distributed Data Processing	2
3	Performance	5
4	Heavily Used Configuration	3
5	Transaction Rate	5
6	Online Data Entry	4
7	End-User Efficiency	3

8	Online Update	5
9	Complex Processing	5
10	Reusability	3
11	Installation Ease	2
12	Operational Ease	3
13	Multiple Sites	1
14	Facilitate Change	3
TDI		**49**

```
VAF = 0.65 + (0.01 × TDI) = 0.65 + 0.49 = 1.14
```

Step 4: Adjusted Function Points (AFP)

```
AFP = UFP × VAF = 239 × 1.14 = 272.5 ≈ 273 Function Points
```

Step 5: Effort & Cost Estimation

Metric	Value
Adjusted FP	273
Effort at 8 hrs/FP	**2,184 person-hours**
Effort at 10 hrs/FP	**2,730 person-hours**
Team Size	2 developers
Duration	~12 weeks = 480 hrs/person
Cost @ ₹500/hr blended rate	**₹10.9L – ₹13.7L**
Productivity	~23 FP/person-month

USE CASE POINTS (UCP)

Step 1: Actor Weighting (UAW)

Actor	Type	Weight	Reason
Payment Gateway	System via API	Simple = 1	REST JSON calls
Fraud Analyst	GUI User	Complex = 3	Full dashboard
Simulator Engine	System via API	Simple = 1	Automated calls
Alert Recipient	External System	Average = 2	Receives notifications
Regulatory System	External via file	Average = 2	Compliance reports
ML Training System	System via API	Average = 2	Kaggle API download

```
UAW = 1 + 3 + 1 + 2 + 2 + 2 = 11
```

Step 2: Use Case Weighting (UUCW)

- Simple (≤ 3 transactions) = 5 pts
- Average (4–7 transactions) = 10 pts
- Complex (> 7 transactions) = 15 pts

#	Use Case	Complexity	Weight
UC1	Evaluate Transaction for Fraud	Complex	15
UC2	Pre-Authorization Decision (BLOCK/CHALLENGE/ALLOW)	Complex	15
UC3	Generate Explainable AI Report	Complex	15
UC4	Detect Fraud Ring via Graph	Complex	15
UC5	Send Multi-Channel Alert	Average	10
UC6	Manage Case (Analyst Workflow)	Average	10
UC7	Train ML Model	Complex	15
UC8	Run Transaction Simulation	Average	10
UC9	View Analytics Dashboard	Average	10
UC10	Configure Risk Thresholds	Simple	5
UC11	Device Fingerprint + Geolocation	Average	10

UC12	Phishing Detection	Average	10
UC13	Merchant Reputation Scoring	Simple	5
UC14	View Performance Metrics	Simple	5

UUCW = 150

UUCP = UAW + UUCW = 11 + 150 = 161

Step 3: Technical Complexity Factor (TCF)

#	Factor	Weight	Rating	Score
T1	Distributed System	2	1	2
T2	Response/Throughput	1	5	5
T3	End-User Efficiency	1	3	3
T4	Complex Internal Processing	1	5	5
T5	Code Reusability	1	3	3
T6	Easy to Install	0.5	2	1
T7	Easy to Use	0.5	3	1.5
T8	Portable	2	2	4
T9	Easy to Change	1	3	3
T10	Concurrent	1	4	4
T11	Special Security	1	5	5
T12	Direct 3rd Party Access	1	3	3
T13	Special User Training	1	2	2
Sum				**41.5**

TCF = 0.6 + (0.01 × 41.5) = 1.015 ≈ 1.02

Step 4: Environmental Factor (ECF)

#	Factor	Weight	Rating	Score
E1	Familiar with dev process	1.5	3	4.5
E2	Application experience	0.5	3	1.5
E3	OO experience	1	4	4
E4	Lead analyst capability	0.5	3	1.5
E5	Motivation	1	4	4
E6	Stable requirements	2	3	6
E7	Part-time workers	−1	2	−2
E8	Difficult programming language	−1	1	−1
Sum				**18.5**

$$ECF = 1.4 + (-0.03 \times 18.5) = 1.4 - 0.555 = 0.845 \approx 0.85$$

Step 5: Final Use Case Points

$$UCP = UUCP \times TCF \times ECF = 161 \times 1.02 \times 0.85 = 139.6 \approx 140$$

Step 6: Effort from UCP

Industry standard: 20 person-hours/UCP

Metric	Value
Final UCP	140
Effort @ 20 hrs/UCP	**2,800 person-hours**
Team Size	2 developers

* * *

1. RISK IDENTIFICATION & CATEGORIES

1.1 Technical Risks

Affect the quality and performance of the software system (technology, design, implementation).

R01: ML Ensemble Inference Latency: Running XGBoost, LightGBM, and Isolation Forest sequentially might exceed the <500ms transaction processing target.

R02: WebSocket Connection Drops: The live simulation dashboard depends entirely on WebSocket stability; drops lead to a frozen UI.

R03: SQLite Database Locking: High-throughput concurrent writes from the simulation loop may cause database locking or corruption.

1.2 Project Risks

Affect the project schedule, cost, resources, and timeline.

R04: **Module Integration Failure**: Combining the Pre-Auth Engine, ML Ensemble, and UI into a cohesive pipeline might exceed the scheduled integration time.

R05: **Local Hardware Constraints**: Training the ML models (especially with SMOTE upsampling on large datasets) might crash local machines due to Out-Of-Memory (OOM) errors.

1.3 Business Risks

Affect the business success of the software project.

R06: Low Fraud Recall (False Negatives): If the ML models miss complex fraud rings, the system loses its primary business value, leading to rejection by financial institutions.

R07: High False Positives: Flagging too many legitimate transactions will cause alert fatigue for analysts and poor user experience.

1.4 Known Risks

Identified in advance during project planning.

R08: Class Imbalance in Training Data: Financial transaction datasets inherently possess extreme class imbalance.

R09: External API Dependency: The system relies on IP geolocation APIs for the Geo-Check module, requiring active internet.

1.5 Predictable Risks

Expected from previous projects or historical data.

R10: Scope Creep: Adding unrequested features (like complex reporting dashboards) may delay the core real-time detection deliverables.

2. RISK PROJECTION AND COMPONENTS

Each risk is evaluated based on its Probability, Impact Level (Negligible, Marginal, Critical, Catastrophic), Impact Component (Performance, Support, Cost, Schedule), and overall Priority.

Risk ID	Risk Description	Component	Probability	Impact Level	Priority
R06	Low Fraud Recall (False Negatives)	Performance	0.40	Catastrophic	1
R01	ML Ensemble Inference Latency	Performance	0.60	Critical	2
R08	Class Imbalance in Training Data	Performance	0.80	Critical	3

R04	Module Integration Failure	Schedule	0.50	Critical	4
R02	WebSocket Connection Drops	Support	0.40	Marginal	5
R03	SQLite Database Locking	Performance	0.30	Critical	6
R05	Local Hardware Constraints	Schedule	0.50	Marginal	7
R07	High False Positives	Support	0.60	Marginal	8
R09	External API Dependency	Performance	0.30	Marginal	9
R10	Scope Creep	Cost/Schedule	0.40	Negligible	10

3. RISK REFINEMENT

Breaking down a complex risk into smaller, manageable risks for better planning.

Refining R01: ML Ensemble Inference Latency

- **Sub-Risk 1:** Feature extraction (e.g., calculating velocity profiles from the database) takes too long.
- **Sub-Risk 2:** LightGBM and XGBoost models running sequentially block the main thread in FastAPI.
- **Sub-Risk 3:** Graph Risk Check traversing too many nodes for dense fraud rings.

4. RISK MITIGATION, MONITORING, AND MANAGEMENT (RMMM)

This section details the strategy (Avoid, Reduce, Transfer, Accept) for the highest priority risks using a proactive risk management approach.

R06: Low Fraud Recall

Strategy: Reduce

Mitigation: Implement an Ensemble approach combining XGBoost, LightGBM, and Isolation Forest. Do not rely on a single model. Tune the decision threshold based on the F1-score rather than raw accuracy. Include anomaly vectors for interpretability.

Monitoring: Continuously track the Confusion Matrix during validation phases.

Management: If recall drops below target, trigger a model retraining phase with adjusted class weights.

R01: ML Ensemble Inference Latency

Strategy: Reduce

Mitigation: Utilize FastAPI's BackgroundTasks for asynchronous scoring where immediate blocking is not strictly required. Cache user behavior profiles in memory to avoid repeated database reads.

Monitoring: Log response times for the transaction endpoint and alert if the average exceeds acceptable limits.

Management: If latency is unacceptable during the demo, temporarily disable the heavy Graph Risk Check and rely solely on the LightGBM/XGBoost models.

R08: Class Imbalance in Training Data

Strategy: Avoid

Mitigation: Apply SMOTE (Synthetic Minority Over-sampling Technique) during the data preprocessing pipeline. Use scale_pos_weight in XGBoost and LightGBM to heavily penalize false negatives.

Monitoring: Review the class distribution of the training batches before finalizing models.

Management: Ensure validation is performed on a strictly un-sampled, real-world distribution to get an accurate representation of performance.

R04: Module Integration Failure

Strategy: Avoid

Mitigation: Define strict JSON schemas and Pydantic models for the data contracts between the Pre-Auth Engine, ML Engine, and Frontend UI.

Monitoring: Conduct end-to-end integration tests using the automated transaction simulator.

Management: If a module fails integration, stub its response (e.g., return a default score of 0 or empty anomaly vector) so the rest of the pipeline can continue functioning.

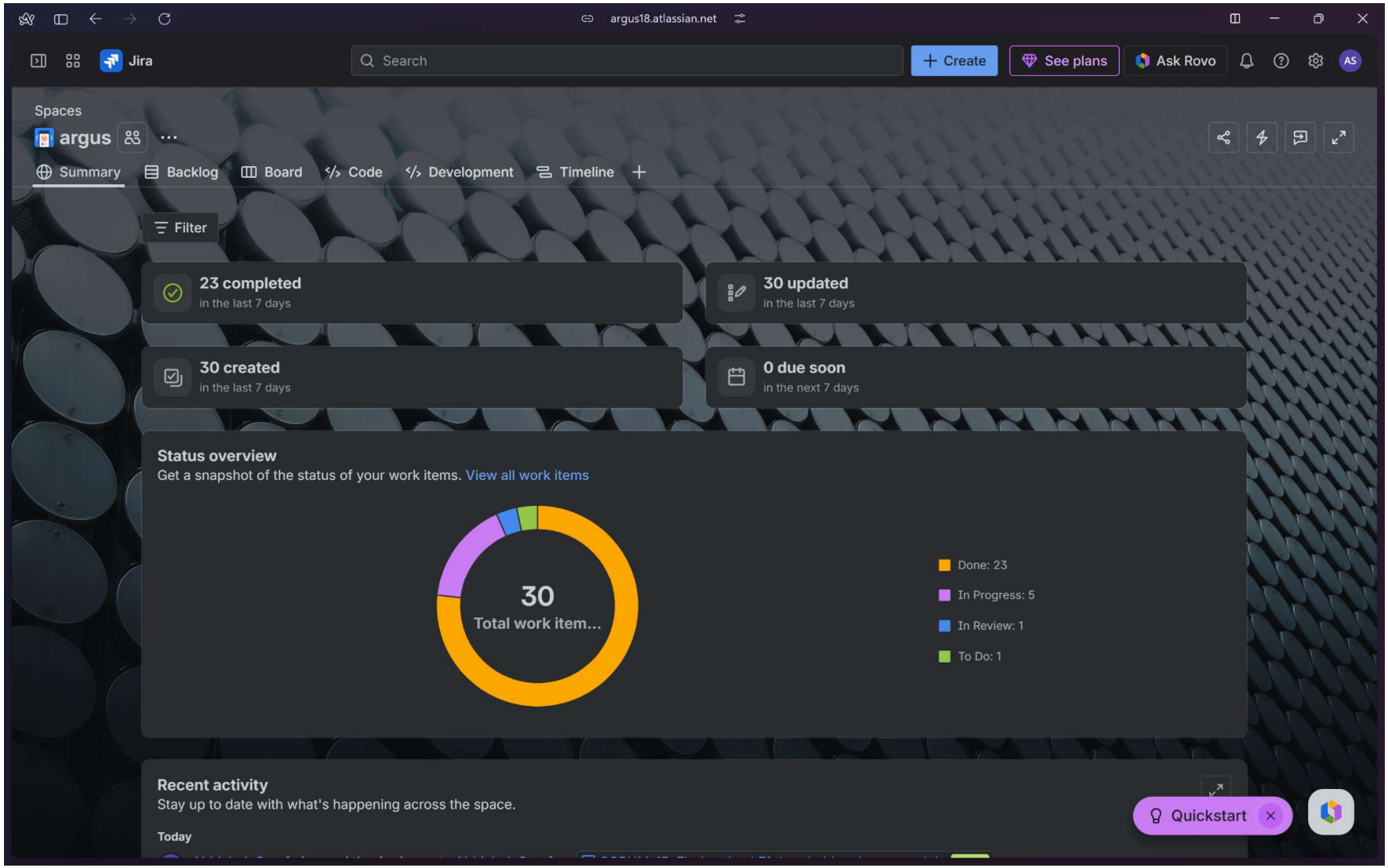
R09: External API Dependency

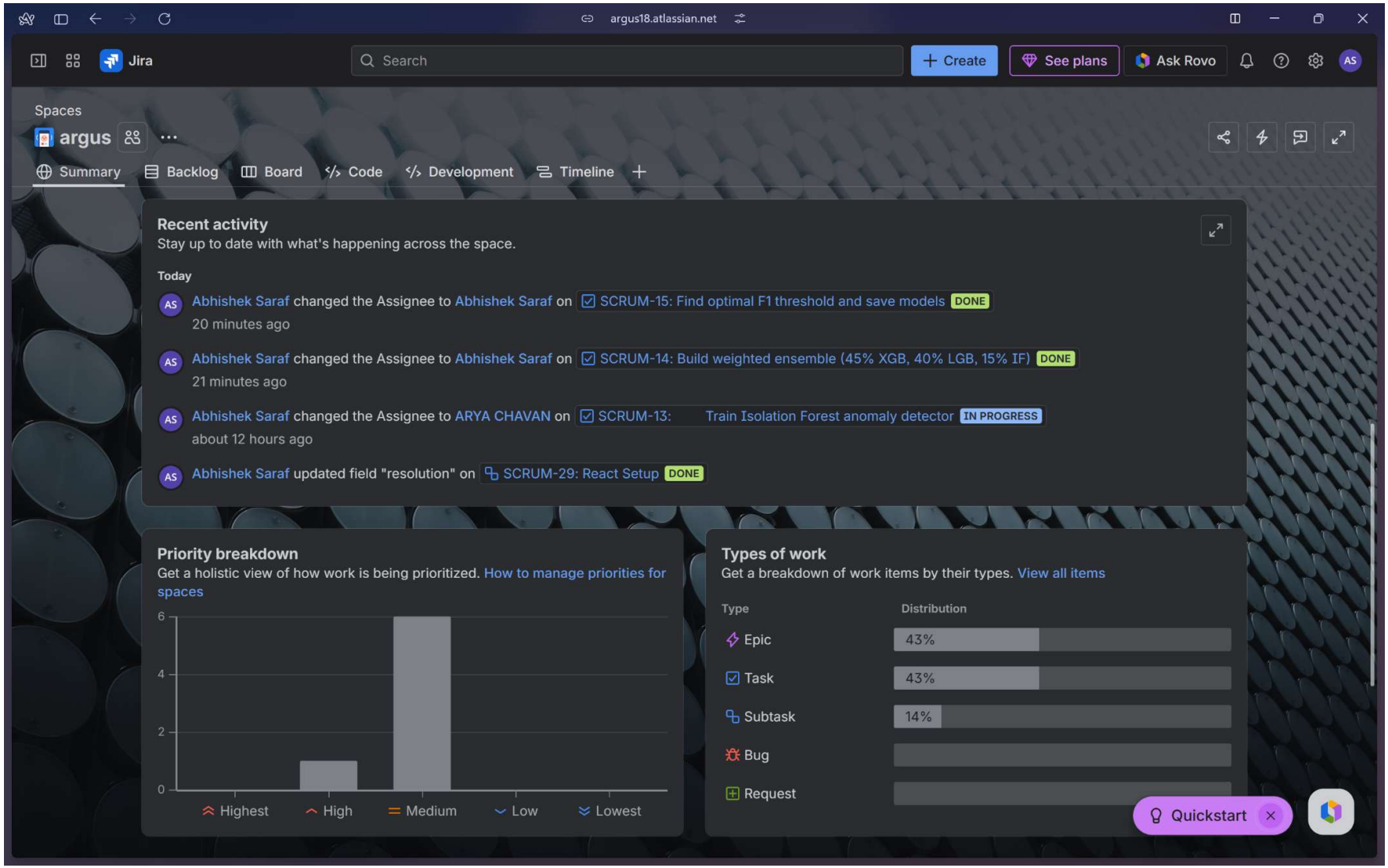
Strategy: Accept

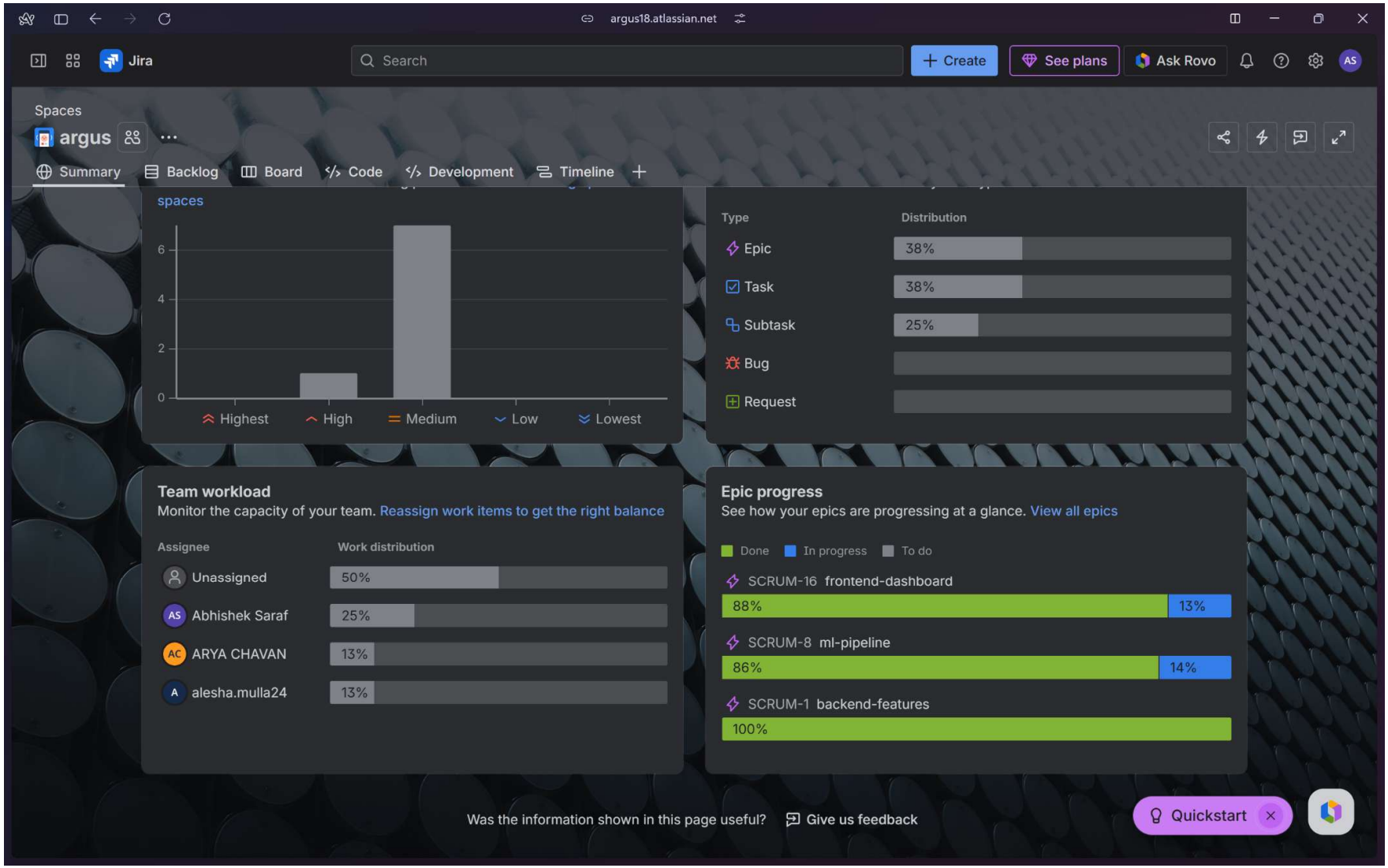
Mitigation: Wrap the IP Geolocation API call in a try/except block with a short timeout.

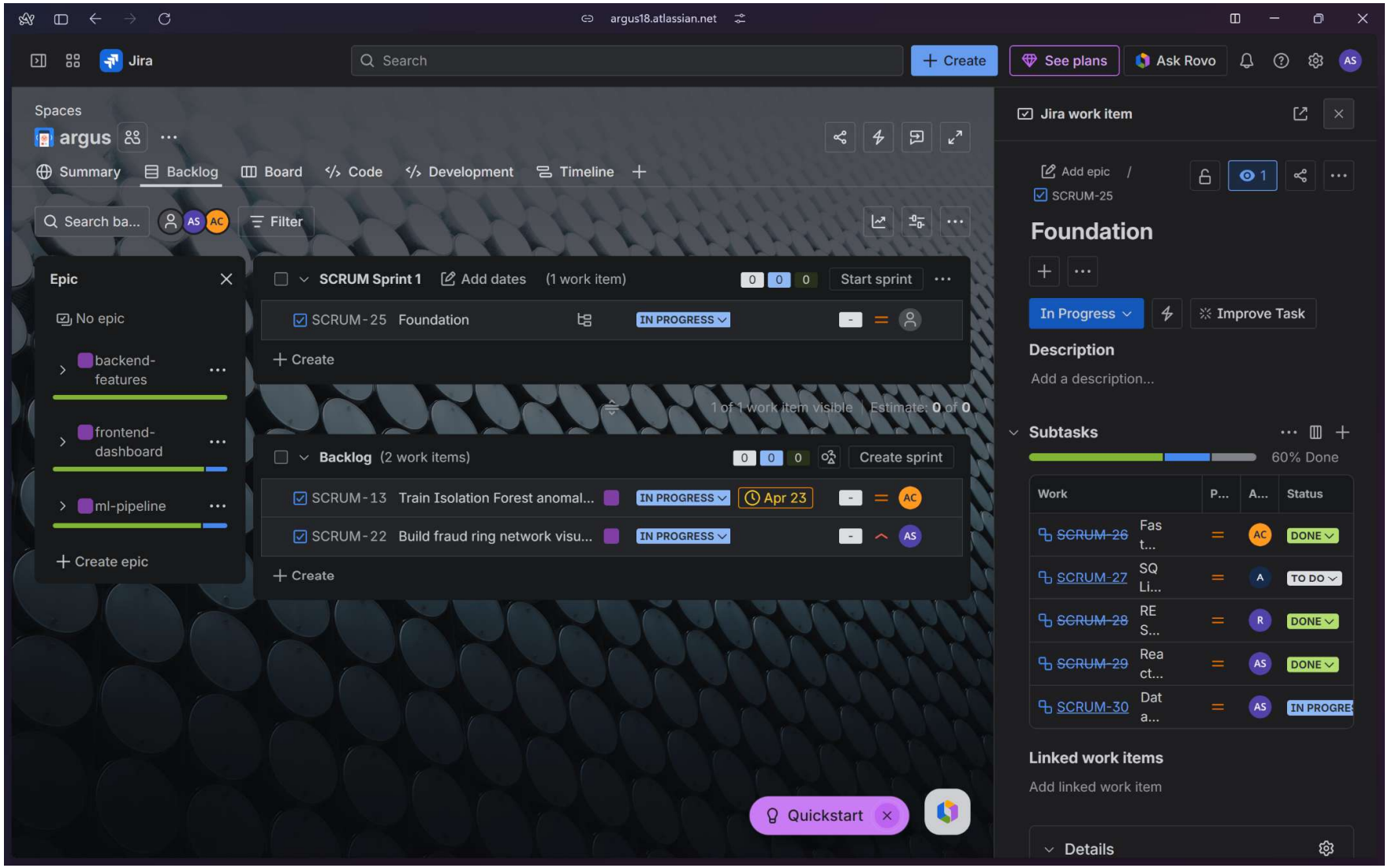
Monitoring: Monitor API failure rates in the terminal logs.

Management: If the hackathon venue lacks internet or the API is down, automatically fallback to returning "Unknown Location" without crashing the transaction pipeline.


argus18.atlassian.net
Jira
Search
Create
See plans
Ask Rovo
AS
Spaces
argus
Summary
Backlog
Board
Code
Development
Timeline
Filter
23 completed
in the last 7 days
30 updated
in the last 7 days
30 created
in the last 7 days
0 due soon
in the next 7 days
Status overview
Get a snapshot of the status of your work items. View all work items
30
Total work item...
Done: 23
In Progress: 5
In Review: 1
To Do: 1
Recent activity
Stay up to date with what's happening across the space.
Today
Quickstart


argus18.atlassian.net
Jira
Search
Create
See plans
Ask Rovo
Spaces
argus
Summary
Backlog
Board
Code
Development
Timeline
Recent activity
Stay up to date with what's happening across the space.
Today
Abhishek Saraf changed the Assignee to Abhishek Saraf on SCRUM-15: Find optimal F1 threshold and save models DONE
20 minutes ago
Abhishek Saraf changed the Assignee to Abhishek Saraf on SCRUM-14: Build weighted ensemble (45% XGB, 40% LGB, 15% IF) DONE
21 minutes ago
Abhishek Saraf changed the Assignee to ARYA CHAVAN on SCRUM-13: Train Isolation Forest anomaly detector IN PROGRESS
about 12 hours ago
Abhishek Saraf updated field "resolution" on SCRUM-29: React Setup DONE
Priority breakdown
Get a holistic view of how work is being prioritized. How to manage priorities for spaces
6
4
2
0
Highest
High
Medium
Low
Lowest
Types of work
Get a breakdown of work items by their types. View all items
Type
Distribution
Epic
43%
Task
43%
Subtask
14%
Bug
Request
Quickstart


argus18.atlassian.net
Jira
Search
Create
See plans
Ask Rovo
AS
Spaces
argus
Summary
Backlog
Board
Code
Development
Timeline
spaces
6
4
2
0
Highest
High
Medium
Low
Lowest
Type
Distribution
Epic
38%
Task
38%
Subtask
25%
Bug
Request
Team workload
Monitor the capacity of your team. Reassign work items to get the right balance
Assignee
Work distribution
Unassigned
50%
Abhishek Saraf
25%
ARYA CHAVAN
13%
alesha.mulla24
13%
Epic progress
See how your epics are progressing at a glance. View all epics
Done
In progress
To do
SCRUM-16 frontend-dashboard
88%
13%
SCRUM-8 ml-pipeline
86%
14%
SCRUM-1 backend-features
100%
Was the information shown in this page useful?
Give us feedback
Quickstart


argus18.atlassian.net
Jira
Search
Create
See plans
Ask Rovo
Spaces
argus
Summary
Backlog
Board
Code
Development
Timeline
Search ba...
Filter
Epic
No epic
backend-features
frontend-dashboard
ml-pipeline
Create epic
SCRUM Sprint 1
Add dates
(1 work item)
Start sprint
SCRUM-25 Foundation
IN PROGRESS
Create
1 of 1 work item visible | Estimate: 0 of 0
Backlog (2 work items)
Create sprint
SCRUM-13 Train Isolation Forest anomal...
Apr 23
SCRUM-22 Build fraud ring network visu...
Create
Quickstart
Jira work item
Add epic
SCRUM-25
Foundation
In Progress
Improve Task
Description
Add a description...
Subtasks
60% Done
Work
P...
A...
Status
SCRUM-26 Fast...
DONE
SCRUM-27 SQ Li...
TO DO
SCRUM-28 RE S...
DONE
SCRUM-29 React...
DONE
SCRUM-30 Data...
IN PROGRES
Linked work items
Add linked work item
Details